

Evidence-Based Developer Reputation for Open-Source Software

Progress presentation: from dataset construction to metric selection evidence

Ali Bakhshayesh

University of Padova

May 9, 2026

Purpose of This Presentation

- Explain the thesis idea in simple language.
- Show what has been built so far, starting from zero.
- Explain how the dataset was created and cleaned.
- Explain the candidate metrics and comparison criteria.
- Show the current result: which metric is strongest for the PoC.
- Clarify what still needs to be done next.

The Big Picture

The thesis is about a simple question:

Main question

Can we create a trustworthy developer-reputation signal from open-source software evidence?

In other words, we want to measure developer activity and responsibility using evidence that can be checked again later.

Why Developer Reputation Matters

Open-source projects often depend on many people:

- some people make small occasional changes;
- some people maintain the project for a long time;
- some people do large technical work;
- some activity may come from bots or automation.

The challenge is to identify useful signals without making unfair or uncheckable claims.

The thesis is connected to a DESTINO-style evidence workflow.

The idea is:

- 1 a developer submits software evidence;
- 2 the evidence is linked to a hash, so it cannot silently change;
- 3 metrics are computed from that evidence;
- 4 compact results are stored in a smart-contract style record;
- 5 another person can later replay the measurement and check the result.

Important Design Choice

We do **not** try to compute repository metrics directly inside a smart contract.

That would be too expensive and too complex because Git history analysis needs:

- repository traversal;
- file and line counting;
- commit and author analysis;
- bot filtering;
- normalization and validation.

Instead, we use a **hybrid design**: compute off-chain, store compact auditable results on-chain.

The Thesis Workflow

The work is organized in two main parts.

Phase I: Build the evidence base

Select repositories, freeze versions, build release windows, extract metrics, clean data, and validate outputs.

Phase II: Compare candidate metrics

Evaluate which metric is stable, understandable, resistant to gaming, useful long-term, and feasible for the PoC.

Where We Started

We first tested the workflow on a small pilot dataset.

- The pilot used 4 repositories.
- The goal was to understand the extraction flow before scaling.
- After the flow worked, we expanded the dataset.

This reduced risk: we tested the method first, then applied it to more projects.

Dataset Expansion

The dataset was expanded from the initial 4 repositories to 10 repositories.

Current repositories:

- `brew`, `click`, `curl`, `flask`, `gin`
- `junit5`, `prettier`, `requests`, `tokio`, `urllib3`

All evidence is now centralized. We no longer depend on separate pilot-only or extended-only CSV files.

What “Dataset Freeze” Means

A dataset freeze means that we fixed the input projects before analysis.

For each repository we recorded:

- project identifier;
- repository URL;
- persistence status;
- software-type tags;
- inclusion metadata.

This is important because later results must be tied to a clear and fixed dataset.

What “Version Freeze” Means

For each repository, we selected the latest 8 stable semantic-version tags.

We excluded unstable release labels such as:

- alpha;
- beta;
- release candidate versions.

Each selected version is tied to an immutable commit hash. This means the exact code state can be checked again later.

Release Windows

After freezing versions, we built release windows.

A release window is the interval between two consecutive tags.

Example:

Simple example

Version 1.0.0 to Version 1.1.0 is one window.

We have:

- 10 repositories;
- 8 frozen tags per repository;
- 7 windows per repository;
- 70 total release windows.

Why Release Windows Matter

We do not only want to know what a project looks like at one moment.

We want to see what changed between two releases:

- how many commits happened;
- how much code changed;
- who contributed;
- whether ownership was concentrated in one or a few developers.

This makes the analysis more useful for developer reputation.

We studied five candidate metric families.

ID	Metric	Simple meaning
M1	Snapshot size	How large the project is at a release tag.
M2	Churn	How many lines were added or deleted in a release window.
M3	Activity cadence	How often developers are active through commits and active days.
M4	Ownership/concentration	How much of the work is done by top contributors.
M5	Responsiveness	How quickly maintainers respond to issues or pull requests.

Important Note About M5

M5, maintenance responsiveness, is useful conceptually.

But it needs issue and pull-request event data.

Our current Phase I dataset focuses on:

- Git tags;
- commits;
- code changes;
- developer identities in Git history.

Therefore, M5 is kept as a future extension, not as a current PoC candidate.

We extracted four implemented metric families from the frozen repositories:

- **M1**: lines of code and source lines of code per frozen tag.
- **M2**: added lines, deleted lines, and total churn per window.
- **M3**: commits, active days, and activity rate.
- **M4**: ownership shares, top contributor shares, and Gini concentration.

These outputs form the empirical base for the later comparison.

Cleaning Developer Identity

Developer identity can be messy.

The same person may appear with different names, but the same email.

We canonicalized developer identity mainly by email. This makes developer-level metrics more consistent.

Cleaning result:

- clean rows remained stable;
- duplicate groups merged: 0;
- zero-commit windows: 0;
- zero-churn windows: 0;
- error log: empty.

Some repository activity comes from bots or automated accounts.

We did not silently mix bot activity with human developer behavior.

Instead:

- bot rows were flagged;
- bot summaries were kept for audit;
- main M3 and M4 evidence uses human-only outputs.

Bots were 15.85% of commits but only 1.61% of churn lines.

Duration Normalization

Release windows are not all the same length.

One project may release after 3 days, another after 6 months.

To compare fairly, we computed normalized values:

- commits per day;
- churn per day;
- churn per KLOC per day.

Very short windows are kept but flagged for sensitivity analysis.

Centralized Evidence

All important Phase I evidence is now centralized in `_all` files.

Examples:

- `dataset_freeze_all.csv`
- `versions_all.csv`
- `windows_all.csv`
- `metrics_window_dev_human_all.csv`
- `ownership_summary_human_all.csv`

This makes the project easier to reproduce and avoids duplicate parallel files.

Phase I produced a clean, reproducible evidence base.

In simple terms:

- we know exactly which repositories are included;
- we know exactly which versions were analyzed;
- we know exactly which release windows were measured;
- we extracted and cleaned the metric evidence;
- we separated human activity from bot activity;
- we normalized values so windows can be compared more fairly.

Why Phase II Was Needed

After Phase I, we had metrics.

But we still needed to answer:

Key question

Which metric is best for a developer-reputation PoC?

A metric is not good just because it is easy to compute. It also needs to be stable, understandable, hard to manipulate, useful over time, and feasible for the target system.

We compared the candidate metrics using four criteria.

ID	Criterion	Simple question
C1	Stability	Does the metric behave consistently over time?
C2	Interpretability	Can we explain the metric clearly?
C3	Gaming resistance	Is the metric hard to manipulate?
C4	Long-term suitability	Does it stay useful as history grows?

Clarification: C1-C4 vs Feasibility

There are two different evaluations.

C1-C4

These ask whether a metric is good as a reputation signal.

Smart-contract feasibility

This asks whether the metric can be implemented and audited safely in the PoC system.

C2, C3, and C4 each use five local dimensions. The feasibility rubric is separate and uses ten dimensions.

C1: Stability

C1 checked how stable the metrics are across release windows.

Main findings:

- M1 is very stable, but it is mostly project context.
- M2 churn is informative but volatile.
- M3 activity has useful rank continuity among recurring developers.
- M4 concentration signals are more stable than raw activity.

C1 did not select the final metric. It produced the first evidence layer.

C2: Interpretability

C2 checked whether each metric is easy to explain.

Metric	Score	Band
M3 Activity cadence	9/10	High
M1 Snapshot size	8/10	High
M2 Churn	8/10	High
M4 Ownership/concentration	7/10	Medium
M5 Responsiveness	6/10	Medium

Main message: M3 is easiest to explain, while M4 is meaningful but needs more explanation.

C3: Gaming Resistance

C3 checked how easy each metric is to manipulate.

Metric	Score	Band
M4 Ownership/concentration	9/10	High
M2 Churn	7/10	Medium
M3 Activity cadence	6/10	Medium
M1 Snapshot size	5/10	Medium
M5 Responsiveness	3/10	Low

Main message: M4 is strongest because sustained ownership is harder to fake than simple commits or churn.

C4: Long-Term Suitability

C4 checked whether the metric remains useful as project history grows.

Metric	Score	Band
M3 Activity cadence	8/10	High
M4 Ownership/concentration	8/10	High
M1 Snapshot size	7/10	Medium
M2 Churn	7/10	Medium
M5 Responsiveness	4/10	Low

Main message: M3 and M4 both work well over longer histories, but they mean different things.

Combined C1-C4 Result

After C1-C4, the strongest implemented reputation-oriented metric is M4.

Metric	C1-C4 average	Role
M4 Ownership/concentration	8.13	Strongest candidate
M1 Snapshot size	7.50	Context only
M3 Activity cadence	7.25	Eligible alternative
M2 Churn	6.50	Supporting signal
M5 Responsiveness	3.25	Future work

After comparing the metrics scientifically, we checked implementation feasibility.

The feasibility rubric asks whether a metric can be:

- computed deterministically;
- stored compactly;
- updated at a realistic frequency;
- audited from hash-pinned evidence;
- represented safely in a smart-contract record;
- protected against simple manipulation.

This rubric has 10 dimensions, D1-D10.

Feasibility Results

Metric	Score	Band	Status
M1 Snapshot size	18/20	High	Context only
M2 Churn	16/20	High	Eligible
M3 Activity cadence	17/20	High	Eligible
M4 Ownership/concentration	16/20	High	Eligible
M5 Responsiveness	10/20	Medium	Excluded

M1 is technically feasible, but it is not selected because size is not a developer-reputation metric by itself.

Final Decision Inputs

The final decision input combines empirical quality and feasibility.

Metric	C1-C4 avg.	Feasibility	Combined
M4 Ownership/concentration	8.13	8.0	8.08
M3 Activity cadence	7.25	8.5	7.75
M2 Churn	6.50	8.0	7.10
M1 Snapshot size	7.50	9.0	Context only
M5 Responsiveness	3.25	5.0	Excluded

Current result: M4 is preferred for the final selection gate.

Why M4 Is Currently the Best Candidate

M4 measures ownership and concentration.

It is strong because:

- it is more reputation-oriented than project size;
- it is less volatile than churn alone;
- it is harder to game than simple commit counts;
- it remains useful over longer histories;
- it can be implemented in the hybrid PoC model.

Main caveat: identity handling must be explicit and auditable.

How M2 and M3 Still Matter

M4 should not be interpreted alone.

Supporting signals:

- **M2 churn** checks whether ownership is backed by real code change.
- **M3 activity cadence** checks whether the developer is active over time.
- **M1 size** gives project context and normalization support.

The PoC may select one primary metric, but the analysis keeps the supporting evidence visible.

What Has Been Committed

The project is now versioned through Git.

Recent committed milestones:

- centralized Phase I ten-repository evidence;
- C1 stability analysis;
- C2 interpretability analysis;
- C3 gaming-resistance analysis;
- C4 long-term suitability analysis;
- smart-contract feasibility scoring.

The branch is ahead of the remote repository because these local commits have not been pushed yet.

What Is New After the Last Clarification

The report now explicitly explains that:

- C2, C3, and C4 each use five criterion-specific dimensions.
- The feasibility rubric is separate and uses ten dimensions.
- These are two different computations with different purposes.

This was added to prevent confusion between C2_D1-C2_D5 and feasibility D1-D10.

At this point:

- Phase I dataset construction is complete.
- Metrics M1-M4 were extracted and cleaned.
- Bot handling and duration normalization are documented.
- Phase II C1-C4 comparison is complete.
- Smart-contract feasibility scoring is complete.
- M4 is the preferred candidate for the final PoC selection gate.

Next Step

The next step is not another metric extraction step.

The next step is:

Final metric selection rationale

Officially record why M4 is selected as the baseline PoC metric, and define exactly how it will be represented in the PoC.

This should freeze:

- selected metric definition;
- analysis policy;
- value encoding;
- evidence hash meaning;
- smart-contract record fields.

After Metric Selection

After the final metric is selected, we can start the PoC implementation.

The PoC should include:

- a minimal smart-contract record structure;
- a compact metric value;
- an evidence hash;
- a submission or registration flow;
- an off-chain verifier that computes the selected metric;
- one or more test examples from the frozen dataset.

- We built a clean ten-repository evidence base.
- We froze versions, windows, and commit hashes for reproducibility.
- We extracted and cleaned M1-M4 metric evidence.
- We evaluated the metrics using C1-C4.
- We applied the smart-contract feasibility rubric.
- The current strongest PoC candidate is M4 ownership/concentration.
- The next step is to write the final selection rationale and then implement the PoC.

Conclusion

The project has moved from raw repository selection to a reproducible evidence base and a justified PoC candidate, with M4 ownership/concentration currently preferred because it gives the best tradeoff between reputation meaning, robustness, long-term usefulness, and implementation feasibility.